Actividad colaborativa II: taller práctico

Pablo Nieto

Programación Orientada a Sitios WEB
Tecnología en Desarrollo de Software para Negocios Digitales, Universidad de Manizales

https://github.com/protonProjects/CafeClima

19 de abril de 2026

Apertura: lunes, 6 de abril de 2026, 00:00
Cierre: domingo, 26 de abril de 2026, 23:55

Orientaciones de desarrollo

Para el desarrollo de esta actividad los estudiantes conformarán equipos de trabajo de máximo tres (03) integrantes. Los conocimientos necesarios son la implementación de bases de datos usando MySQL, la implementación del modelo cliente-servidor y la manipulación de datos en la base de datos usando el lenguaje de programación PHP. Los pasos a seguir se encuentran en el archivo anexo a la actividad.

Criterios de evaluación

1. Funcionalidad del sistema acorde con lo solicitado en el enunciado.
2. Uso de conceptos de vistos durante el desarrollo de la unidad.
3. Presentación Del documento a entregar.
4. Creatividad y uso de buenas prácticas de desarrollo.

1. Introducción

En el Taller 1 construimos CaféClima, una app en Angular que le dice al caficultor si va a llover y a cuánto está el café, consultando dos APIs externas. Eso resolvió el problema de la información en tiempo real, pero los datos no quedaban guardados en ningún lado, cada vez que el campesino abría la app la información desaparecía

En este taller le añadimos memoria a la app. Ahora CaféClima tiene un backend propio hecho en PHP que corre sobre XAMPP, con una base de datos MySQL que guarda el historial de cada consulta de clima y cada precio registrado. También permite administrar los municipios desde donde se consulta el clima, con un CRUD completo.

2. Objetivo General del proyecto

Añadir persistencia de datos a CaféClima mediante un backend REST en PHP conectado a MySQL, de manera que cada consulta de clima y precio quede registrada y el caficultor pueda llevar un historial de las condiciones de su región.

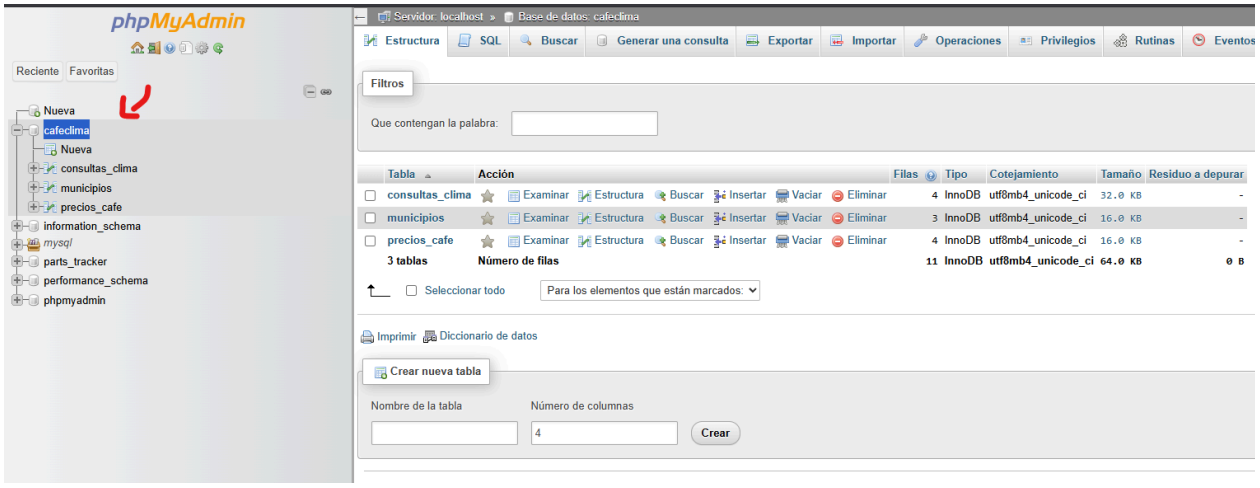
3. **Desarrollo de la actividad propuesta**

4. **Stack técnico**

Frontend	Angular 21 (existente del Taller 1)
Backend / API REST	PHP 8.x sobre XAMPP Apache
Base de datos	MySQL 8.x — phpMyAdmin
Servidor local	XAMPP — Apache en `localhost:80`, MySQL en `3306`

5. **Base de datos — 3 tablas**

La base de datos `cafeclima` tiene tres tablas relacionadas entre sí.




```
sql CREATE TABLE municipios (
        id          INT AUTO_INCREMENT PRIMARY KEY,
        nombre      VARCHAR(100) NOT NULL,
        departamento  VARCHAR(100) NOT NULL,
        activo      TINYINT(1)   NOT NULL DEFAULT 1,
        creado_en   TIMESTAMP    DEFAULT CURRENT_TIMESTAMP
);

CREATE TABLE consultas_clima (
        id              INT AUTO_INCREMENT PRIMARY KEY,
        id_municipio    INT         NOT NULL,
        probabilidad_lluvia INT     NOT NULL,
        descripcion     VARCHAR(255) NOT NULL,
        temperatura     DECIMAL(5,2) NOT NULL,
```

```
	consultado_en	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP,
	FOREIGN KEY (id_municipio) REFERENCES municipios(id) ON DELETE CASCADE
);


CREATE TABLE precios_cafe (
	id		INT AUTO_INCREMENT PRIMARY KEY,
	precio_usd	DECIMAL(10,4) NOT NULL,
	precio_cop	DECIMAL(15,2) NOT NULL,
	tasa_cambio	DECIMAL(10,4) NOT NULL,
	registrado_en TIMESTAMP	DEFAULT CURRENT_TIMESTAMP
);
```

La tabla `municipios` tiene los pueblos desde donde se consulta el clima. `consultas_clima` guarda cada vez que la app llama a OpenWeatherMap, con la llave foránea `id_municipio` que la relaciona con la tabla anterior. `precios_cafe` guarda cada consulta de tasa de cambio con el precio calculado en pesos.

6. Estructura del backend

api/config/db.php ------> Conexión PDO (singleton)

api/municipios/index.php ------> CRUD: GET · POST · PUT · DELETE

api/consultas/index.php ------> Historial de clima: GET · POST

api/precios/index.php ------> Historial de precios: GET · POST

7. Conexión a la base de datos — `api/config/db.php`

La conexión usa PDO con el patrón singleton para no abrir múltiples conexiones. Se configuró con `ERRMODE_EXCEPTION` para que los errores de MySQL se conviertan en excepciones PHP que podemos capturar con `try/catch`.

8. Endpoints implementados

GET `/cafeclima/api/municipios/` Listar municipios activos 200

POST `/cafeclima/api/municipios/` Crear municipio 201

PUT `/cafeclima/api/municipios/?id={id}` Actualizar nombre 200 / 404
DELETE `/cafeclima/api/municipios/?id={id}` Eliminar municipio 200 / 404

GET `/cafeclima/api/consultas/` Historial de clima (con JOIN) 200

POST `/cafeclima/api/consultas/` Guardar consulta 201

GET `/cafeclima/api/precios/` Historial de precios 200

POST `/cafeclima/api/precios/` Guardar precio 201

Los códigos de error manejados son `400` (body inválido), `404` (registro no existe) y `500` (error de BD).

9. Cambios en Angular — `src/app/services/data.ts`

Se añadieron 7 métodos al `DataService` existente para consumir la API local:

- `getMunicipios()` — lista los municipios
- `crearMunicipio(nombre, departamento)` — crea uno nuevo
- `eliminarMunicipio(id)` — lo elimina
- `guardarConsulta(payload)` — guarda el resultado del clima
- `getHistorialConsultas()` — trae el historial
- `guardarPrecio(payload)` — guarda el precio calculado
- `getHistorialPrecios()` — trae el historial de precios

También se modificó `getPrecioCafeEnCOP()` para que retorne no solo el texto formateado sino también los valores numéricos (`precioCop`, `trmActual`) que se necesitan para guardar en la BD.

10. Integración en HomeComponent — `src/app/pages/home/home.ts`

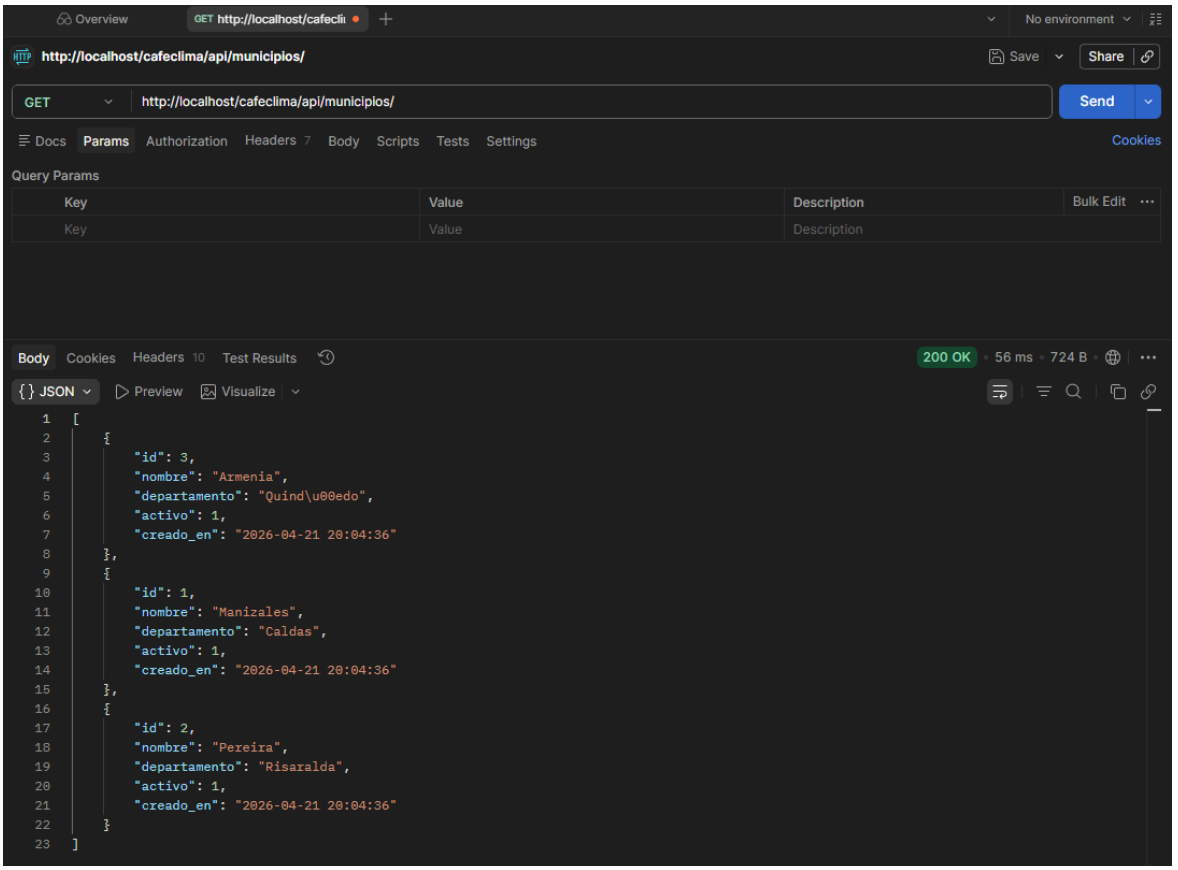
El `HomeComponent` ahora llama a `guardarConsulta()` en el `next:` del observable del clima, justo después de recibir la respuesta de OpenWeatherMap, y a `guardarPrecio()` en el `next:` del observable de ExchangeRate. Esto pasa de forma automática cada vez que el caficultor abre la app.

11. Pruebas en Postman

Se hicieron 9 llamados para verificar que todos los endpoints respondieran correctamente antes de dar por terminado el backend.

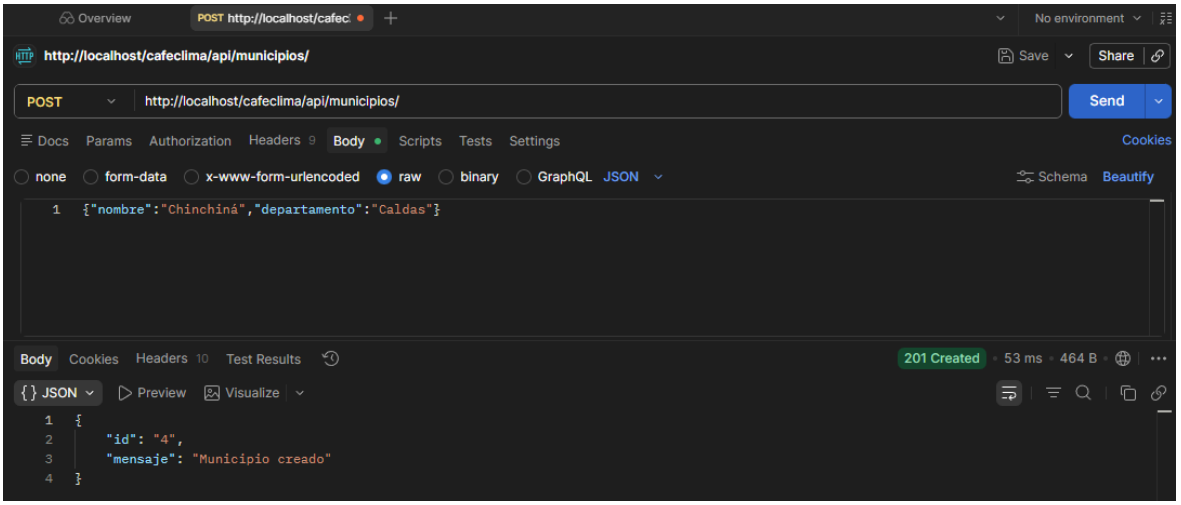
Llamado 1 — GET municipios

URL: `http://localhost/cafeclima/api/municipios/`
Respuesta: 200 OK, array con Manizales, Pereira y Armenia



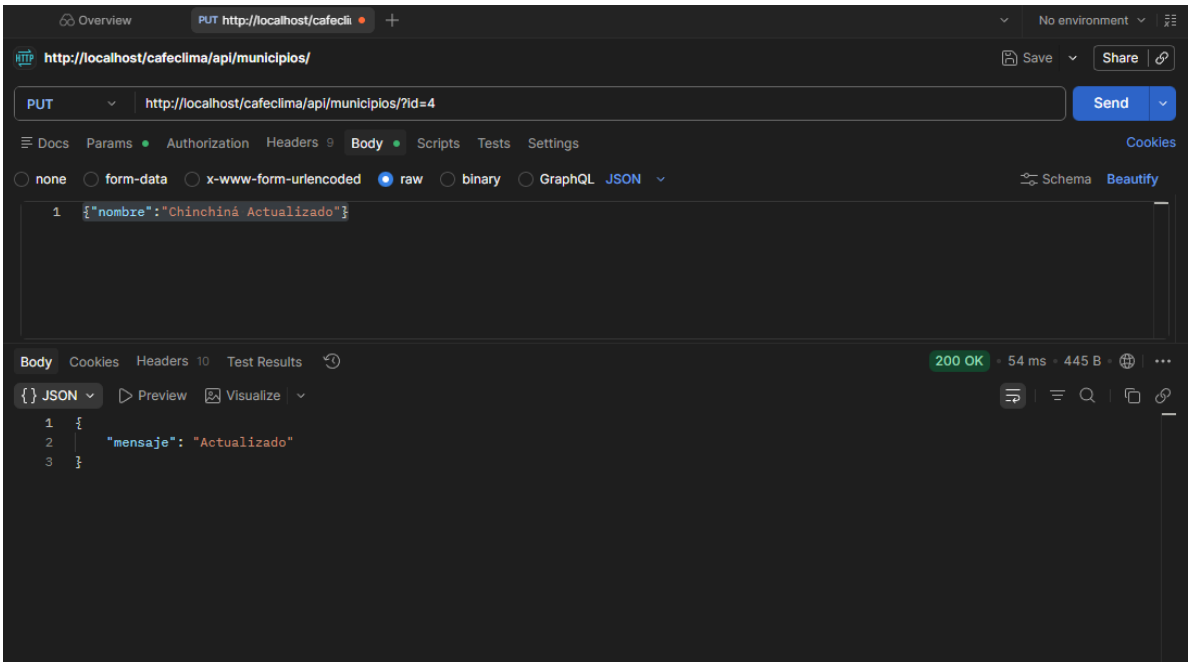
Llamado 2 — POST municipio

URL: `http://localhost/cafeclima/api/municipios/`
Body: `{"nombre":"Chinchiná","departamento":"Caldas"}`
Respuesta: 201 Created



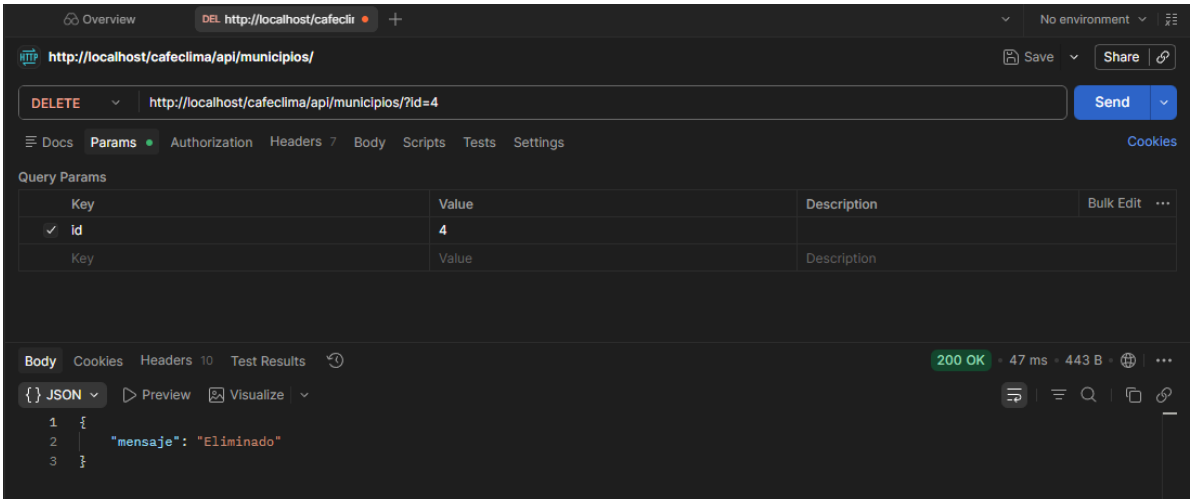
Llamado 3 — PUT municipio

URL: `http://localhost/cafeclima/api/municipios/?id=4`
Body: `{"nombre":"Chinchiná Actualizado"}`
Respuesta: 200 OK



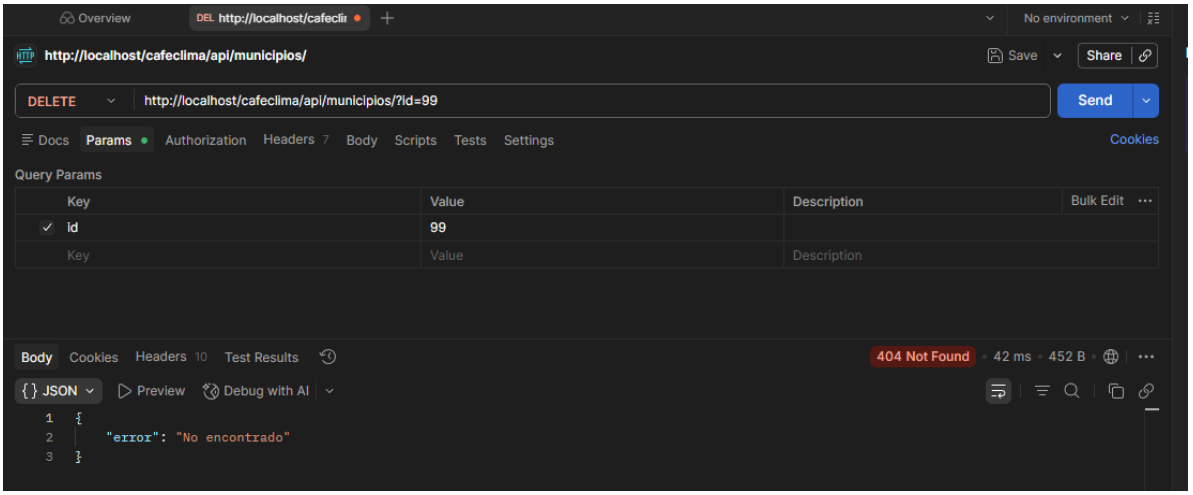
Llamado 4 — DELETE municipio (existente)

URL: `http://localhost/cafeclima/api/municipios/?id=4`
Respuesta: 200 OK



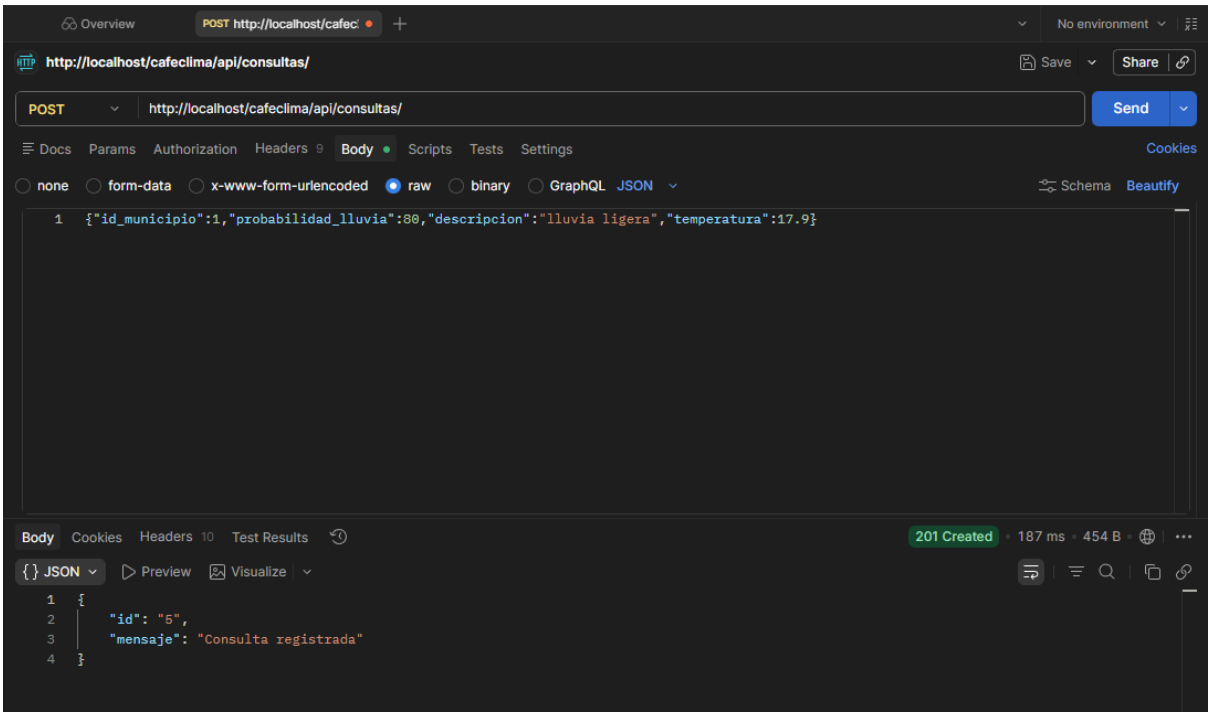
Llamado 5 — DELETE municipio (inexistente)

URL: `http://localhost/cafeclima/api/municipios/?id=99`
Respuesta: 404 Not Found



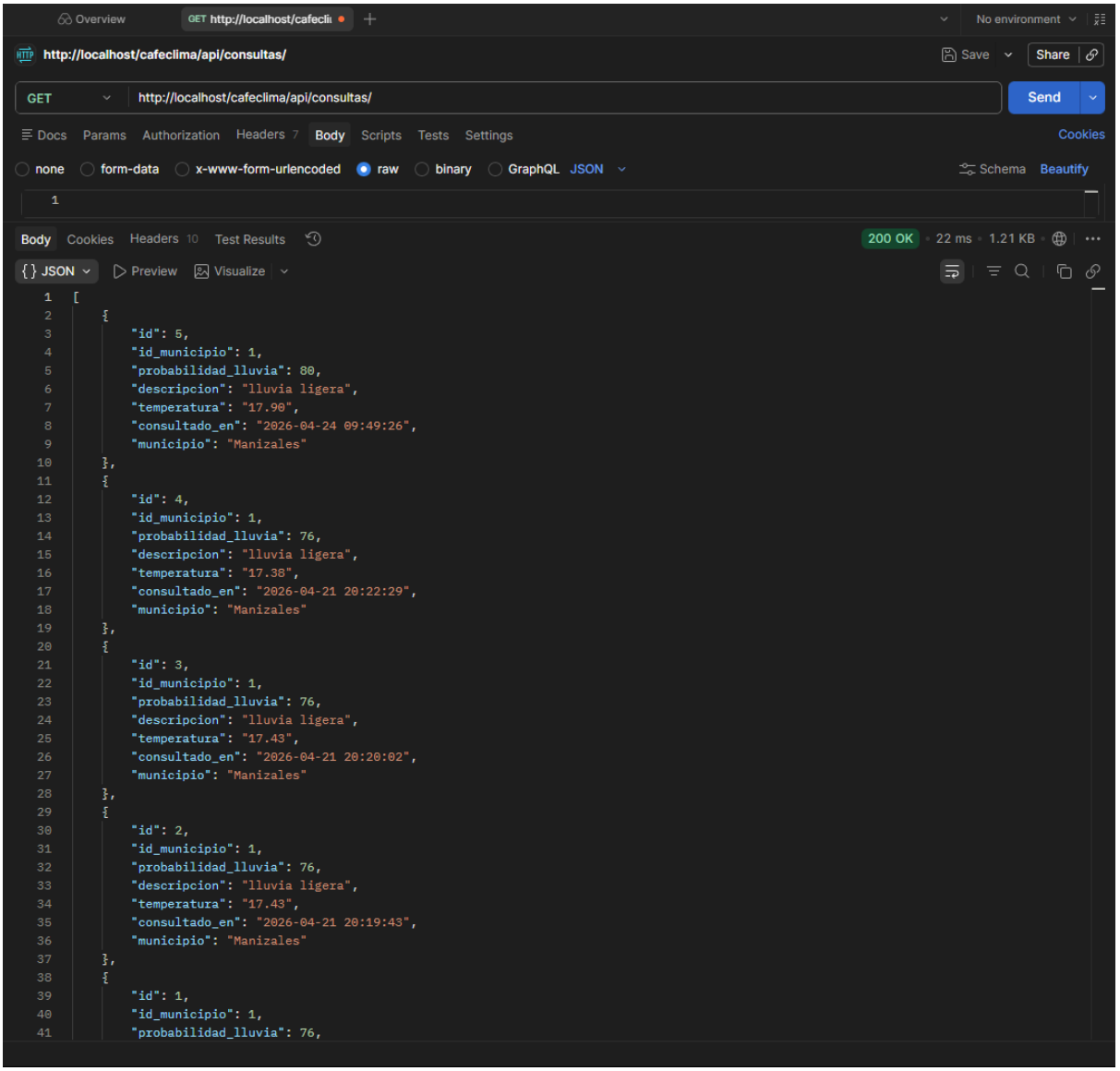
Llamado 6 — POST consulta

URL: `http://localhost/cafeclima/api/consultas/`
Body: `{"id_municipio":1,"probabilidad_lluvia":80,"descripcion":"lluvia ligera","temperatura":17.9}`
Respuesta: 201 Created



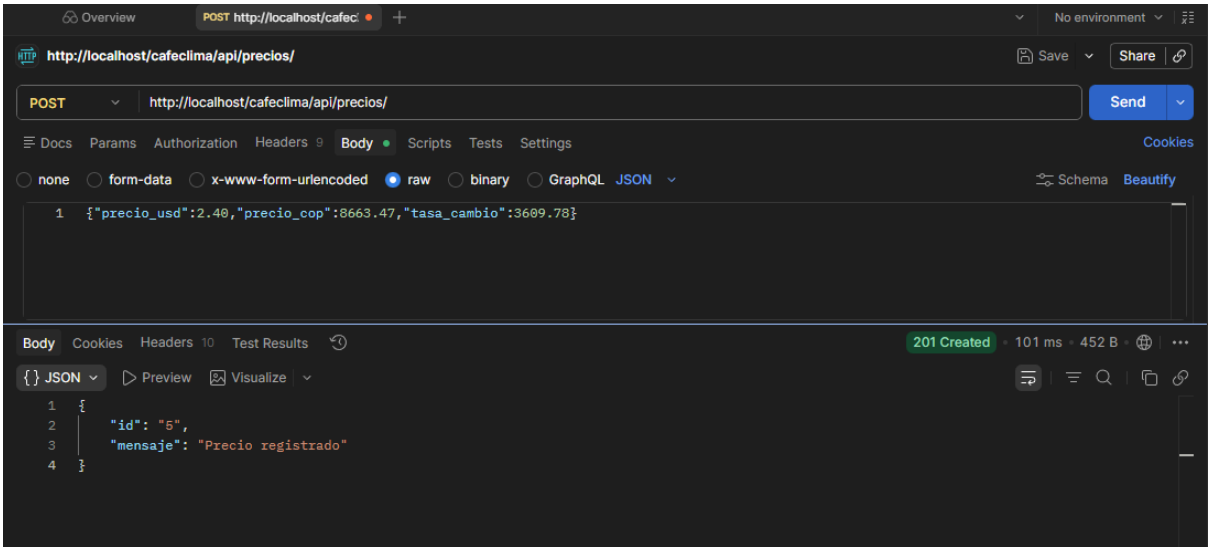
Llamado 7 — GET consultas

URL: `http://localhost/cafeclima/api/consultas/`
Respuesta: 200 OK, array con historial y JOIN al nombre del municipio



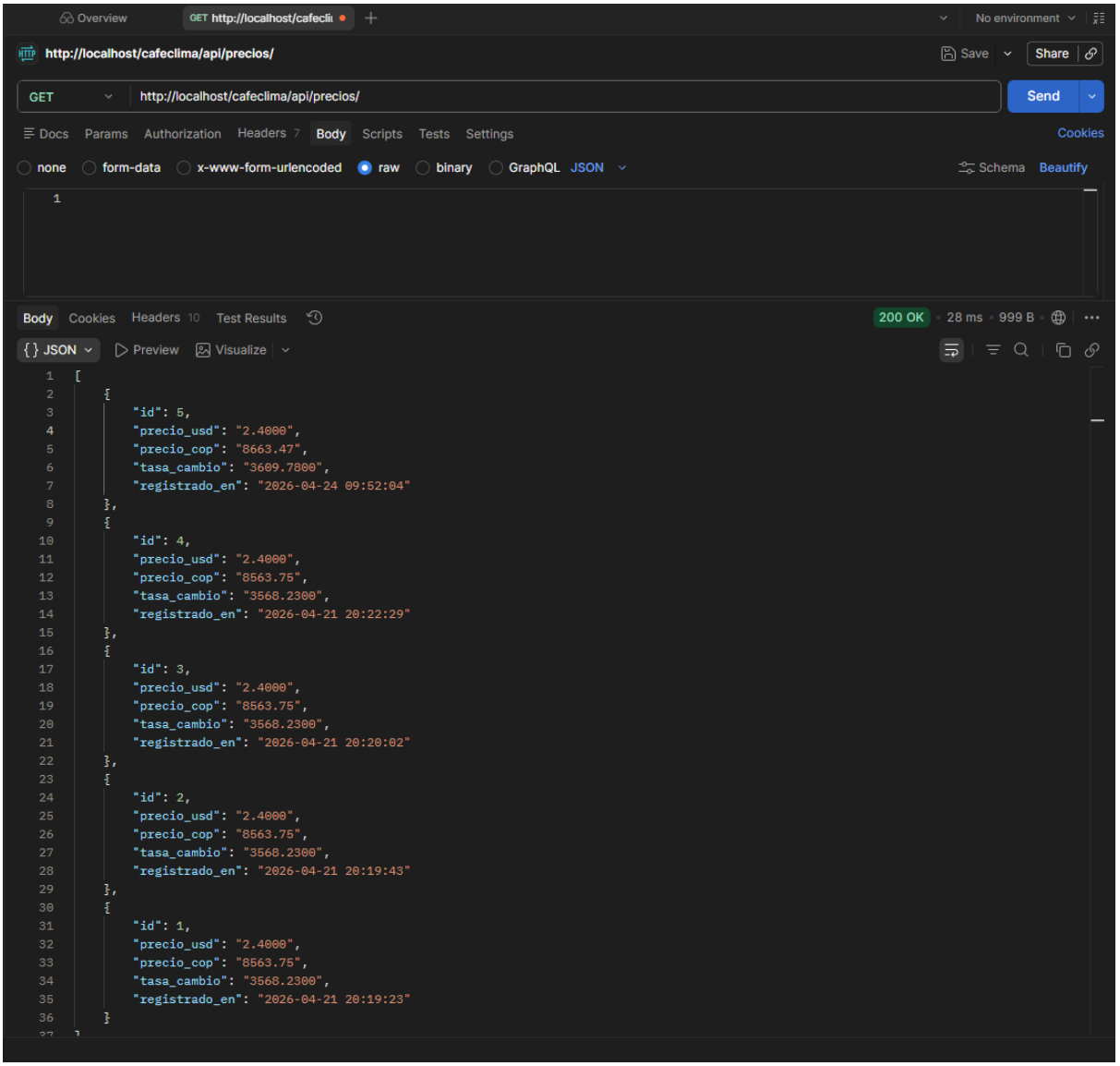
Llamado 8 — POST precio

URL: `http://localhost/cafeclima/api/precios/`
Body: `{"precio_usd":2.40,"precio_cop":8663.47,"tasa_cambio":3609.78}`
Respuesta: 201 Created



Llamado 9 — GET precios

URL: `http://localhost/cafeclima/api/precios/`
Respuesta: 200 OK, historial de precios registrados



12. Conclusiones:

Con PHP y MySQL le dimos a CaféClima algo que le faltaba: memoria. Antes la app mostraba los datos pero no los guardaba, ahora cada vez que el caficultor revisa el clima o el precio ese dato queda en la base de datos. El modelo cliente-servidor quedó claro en la práctica: Angular hace las peticiones, PHP las recibe y las procesa, MySQL las persiste.

Lo más valioso fue ver cómo las dos partes se conectan, el frontend que ya teníamos del taller anterior no cambió en su estructura, solo se añadieron llamadas al backend propio en el mismo momento en que llegan los datos de las APIs externas.

https://github.com/protonProjects/CafeClima
https://cafeclima.vercel.app/